

STTAURX

Database Implementation Plan

Phase 1: Users, Wallets, Transactions

Document Status	PLAN - Awaiting Approval
Date	February 3, 2026
Version	1.0
Author	Development Team

Executive Summary

This plan addresses three critical questions before implementing the STTAURX database infrastructure:

- **Mock Data Testing** - How to support both mock mode and database mode
- **Rust Compatibility** - Integration between Rust blockchain core and Node.js API
- **Security Protocols** - SQL injection prevention and comprehensive security

Question 1: Mock Data Testing

Problem:

The current server.js uses in-memory mock data. We need both mock mode for quick testing and database mode for production.

Solution: Hybrid Architecture with Feature Flags

```
USE_DATABASE=false → Mock mode (in-memory data)
```

```
USE_DATABASE=true → Database mode (PostgreSQL + Redis)
```

Benefits:

- ✓ Zero code changes to switch modes
- ✓ Test locally without database setup
- ✓ Same API contract for both modes
- ✓ Easy to add new services

Question 2: Rust Compatibility

Current Architecture:

Your Rust crates use RocksDB for blockchain storage (blocks, state, transactions). This is correct for blockchain data.

The PostgreSQL database is for the Node.js API layer - user accounts, marketplace orders, KYC, etc. These are application-level concerns, not blockchain-level.

Recommended Approach (Phase 1):

Keep Node.js for application API, Rust for blockchain core. They can sync via HTTP/gRPC.

Future Option:

Add SQLx to Rust for shared DB access if needed. SQLx provides async, compile-time checked queries.

Question 3: Security Protocols

Security Architecture Overview - 6 Layers:

Layer	Components
1. Network	HTTPS/TLS, Firewall, Rate limiting
2. Input	Zod schema validation, Sanitization
3. Database	Prisma parameterized queries (SQL injection proof)
4. Auth	JWT + bcrypt (cost 12) + Session management
5. Authorization	Role-based access control (RBAC)
6. Audit	Activity logging for all sensitive operations

SQL Injection Prevention:

Prisma ORM automatically uses parameterized queries, preventing SQL injection attacks:

```
// Prisma parameterized query (SAFE)
const user = await prisma.user.findUnique({
  where: { email: userInput } // Auto-parameterized
});
```

Implementation Phases

Phase	Timeline	Tasks
1A: Mock Mode	1 day	Service factory pattern, USE_DATABASE flag
1B: Database	2-3 days	PostgreSQL + Redis setup, Prisma migrations
1C: Security	1-2 days	Auth middleware, JWT, RBAC, audit logging

Decision Points (User Selections)

Decision	Selected Option
Default Mode	Mock mode (USE_DATABASE=false)
Authentication	JWT + Refresh Tokens (15min/7d)
Database Location	Local on Droplet (164.92.116.28)

Files Created

File	Purpose
database/schema.prisma	Phase 1 database schema (11 tables)
database/db.js	PostgreSQL + Redis connection layer
database/setup-droplet.sh	Auto-install script for droplet
database/seeds/seed.js	Demo data seeder
.env.example	Environment template
DATABASE_SETUP.md	Deployment guide

Phase 1 Schema (11 Tables)

Table	Purpose
users	User accounts + auth
user_sessions	Session management
kyc_documents	KYC verification
wallets	User wallets
wallet_balance_history	Balance audit trail
blocks	Blockchain blocks
transactions	All tx types
transaction_receipts	Receipts
audit_logs	Admin activity
system_settings	Platform config
fee_config	Fee settings

Approval Checklist

- Overall approach is acceptable
- Decision points confirmed
- Phase 1A: Service factory pattern
- Phase 1B: Database integration
- Phase 1C: Security hardening
- Ready to proceed with implementation

Document generated: February 03, 2026 at 23:17

STTAURX Gold Stablecoin Platform